

GOOGLE CAREER / INTERVIEW BLUEPRINT

DAY 4 — DETAILED OBJECTIVES & EXECUTION PLAN

Sunday, 13 September 2026

Day 4 Theme: From a single working API → a properly structured backend service

Total Target: 3 hours

Primary Goal: Expand **ai-engineering-intelligence-platform** into a small but complete document API with CRUD-style operations, stronger service/repository separation, Pydantic models, automated tests, and the next DSA pattern: Sliding Window.

Day 3 context: Day 3 was used to study and revise Day 1 and Day 2. Day 4 therefore moves forward instead of repeating the earlier study work.

Sequencing: PostgreSQL remains intentionally postponed for Day 5. Day 4 strengthens the application boundary first so database integration can be added without mixing persistence concerns into the API or business logic.

1. Day 4 Objectives

By the end of Day 4, the objective is to move from a minimal FastAPI service to a small, coherent backend service while preserving the architecture established on Day 2.

Backend

- ☐ Review the existing FastAPI application without redoing the full Day 1/Day 2 study.
- ☐ Verify that GET /, GET /health and POST /documents still work.
- ☐ Add document retrieval for a single document.
- ☐ Add document listing.
- ☐ Add document deletion.
- ☐ Introduce document-ID-based API behavior.
- ☐ Implement GET /documents.
- ☐ Implement GET /documents/{id}.
- ☐ Implement DELETE /documents/{id}.
- ☐ Keep API → Service → Repository separation.
- ☐ Keep business rules inside DocumentService.
- ☐ Keep storage operations inside DocumentRepository.
- ☐ Continue using in-memory storage.
- ☐ Use explicit Pydantic request/response models.
- ☐ Handle missing documents as HTTP 404.
- ☐ Expand automated API tests for the new behavior.

DSA

- ☐ Learn the Sliding Window pattern.
- ☐ Understand fixed-size versus variable-size windows.
- ☐ Solve Longest Substring Without Repeating Characters.
- ☐ Solve Maximum Average Subarray I.
- ☐ Solve Minimum Size Subarray Sum.
- ☐ For every problem, understand the brute-force approach.
- ☐ For every problem, identify why Sliding Window applies.
- ☐ For every problem, derive the optimized approach.
- ☐ For every problem, explain the invariant.
- ☐ For every problem, record time and space complexity.
- ☐ Record the mistake or observation you want to remember.
- ☐ Explain each solution without looking at the code.

Engineering

- ☐ Use meaningful Git commits.
- ☐ Update the README with current API capabilities and architecture.
- ☐ Push the completed work to GitHub.
- ☐ Understand what the repository boundary enables when PostgreSQL is introduced later.
- ☐ Avoid premature infrastructure and AI-stack expansion.

2. Day 4 Time-Blocked Execution Plan

Block	Time	Work	Expected outcome
1	25 min	Backend revision	Understand current architecture and existing Day 2 API
2	45 min	FastAPI + Pydantic expansion	CRUD-style document API
3	35 min	Service + Repository	Clean retrieval/list/delete flow
4	35 min	API testing	Tests for Day 4 behavior
5	30 min	DSA — Sliding Window	3 problems + Big-O + observations
6	10 min	Git + documentation	Clean commits + README update

3. Block 1 — Backend Revision

Time: 25 minutes

Do not reread the entire Day 1 and Day 2 material. Day 3 was your consolidation day. Use your actual project and reconstruct the architecture from memory.

Task 1 — Draw the architecture

```
HTTP Client
  ↓
FastAPI
  ↓
API Layer
  ↓
DocumentService
  ↓
DocumentRepository
  ↓
In-Memory Storage
```

Task 2 — Explain POST /documents

```
POST /documents
  ↓
Pydantic validation
  ↓
API layer
  ↓
DocumentService
  ↓
DocumentRepository
  ↓
In-memory store
  ↓
DocumentResponse
  ↓
HTTP response
```

Self-check

- ☐ Where does request/schema validation happen?
- ☐ Where does business logic happen?
- ☐ Where does storage happen?
- ☐ Where does HTTP-specific behavior happen?
- ☐ Why does the API call the service instead of directly calling the repository?
- ☐ What would break if business logic were placed inside the route?

4. Block 2 — Expand FastAPI + Pydantic

Time: 45 minutes

Day 2 established POST /documents. Day 4 makes the document API useful by adding retrieval, listing and deletion.

4.1 GET /documents

GET /documents

```
[
  {
    "id": "doc-001",
    "name": "engine-requirement",
    "document_type": "requirement",
    "version": "1.0",
    "status": "created"
  }
]
```

Required flow:

```
GET /documents
  ↓
API
  ↓
DocumentService
  ↓
DocumentRepository
  ↓
In-memory storage
```

Key objective: the API should not bypass the service layer just because listing is simple.

4.2 GET /documents/{id}

GET /documents/doc-001

```
{
  "id": "doc-001",
  "name": "engine-requirement",
  "document_type": "requirement",
  "version": "1.0",
  "status": "created"
}
```

Missing-resource flow:

```
DocumentRepository
  ↓
DocumentNotFoundError
  ↓
API error handling
  ↓
HTTP 404
```

4.3 DELETE /documents/{id}

DELETE /documents/doc-001

Keep the same architectural flow:

```
HTTP
  ↓
API
  ↓
Service
  ↓
Repository
  ↓
Storage
```

Do not put deletion business behavior directly in the FastAPI route.

4.4 Pydantic model review

```
DocumentCreate
  ↓
Client → API

DocumentResponse
  ↓
API → Client

API Schema
  ≠
Business Object
  ≠
Database Model
```

The Day 2 foundation established that API schemas define the HTTP boundary, business objects represent domain/business logic, and database models represent persistence.

4.5 Questions to answer while coding

- [] Why is GET /documents a collection endpoint?
- [] What should an empty document collection return?
- [] Why does GET /documents/{id} need 404 behavior?
- [] Why should DELETE /documents/{id} call DocumentService?
- [] Why should the response be represented by an explicit schema?
- [] Which fields are client-supplied versus server-generated?
- [] Which changes belong to the API model and which belong to the business object?

5. Block 3 — Service + Repository

Time: 35 minutes

This block reinforces the architecture rather than introducing another layer.

Repository responsibilities

```
create()
get()
get_all()
delete()
```

Service responsibilities

```
create_document()
get_document()
list_documents()
delete_document()
```

API responsibilities

```
Routing
Request validation
Response models
HTTP status codes
HTTP error translation
```

Repository: owns storage operations and should not decide HTTP status codes.

Service: owns business behavior and should not become coupled to FastAPI or HTTP response objects.

Architecture self-check

- [] Can you replace the repository implementation without rewriting the API contract?
- [] Can you call the service from something other than HTTP?
- [] Can you test the service without starting FastAPI?

- [] Can you test API behavior without putting business rules in routes?
- [] Can you explain why in-memory storage is acceptable at this stage?

Day 4 architectural objective: make the repository boundary useful before Day 5 introduces durable database persistence.

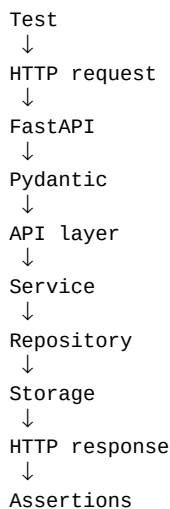
6. Block 4 — API Testing

Time: 35 minutes

Expand the Day 2 test suite so that the new API behavior is executable and repeatable.

Behavior	Expected
GET /	Success response
GET /health	Success + healthy status
POST /documents	Valid request creates/returns document
Invalid POST /documents	Validation/client error
GET /documents	Success + list
GET /documents/{id}	Existing document returned
GET /documents/{id}	Missing document → 404
DELETE /documents/{id}	Existing document deleted successfully
DELETE /documents/{id}	Missing document → 404

Testing mental model



The key lesson is that API behavior is encoded in tests so it can be run repeatedly instead of verified only by manually clicking through Swagger.

7. Block 5 — DSA: Sliding Window

Time: 30 minutes

Day 4 introduces Sliding Window as the next major pattern after the Day 1 arrays/hash maps work and Day 2 Two Pointers.

7.1 Pattern definition

Sliding Window is useful for problems involving a contiguous range of an array or string. Instead of recomputing every possible range from scratch, maintain a current window and move its boundaries as the scan progresses.

```
left
↓
[a b c d e f g]
      ↑
      right
```

7.2 Pattern recognition

Ask whether the problem is about a contiguous subarray or substring and asks for a longest, shortest, maximum, minimum, or fixed-size result. If so, Sliding Window should be considered.

7.3 Fixed-size window

The window size is known, usually k .

```
[a b c d] e f g
  ↓ slide
a [b c d e] f g
```

Instead of recalculating every sum:

```
new_sum = old_sum - outgoing + incoming
```

Day 4 fixed-window problem: Maximum Average Subarray I.

7.4 Variable-size window

The window expands and contracts according to a condition.

```
expand right → condition becomes valid
shrink left  → condition remains/changes toward target
repeat       → track best answer
```

Day 4 variable-window problems: Longest Substring Without Repeating Characters and Minimum Size Subarray Sum.

8. DSA Problem 1 — Longest Substring Without Repeating Characters

Goal: Find the length of the longest substring without repeating characters.

Example:
abcabcbb

Longest valid substring:
abc

Answer:
3

Tasks

- [] Write the brute-force approach.
- [] Explain why repeated substring work is wasteful.
- [] Define the current window.
- [] Track the characters required to maintain uniqueness.
- [] Expand the right side as you scan.
- [] When a duplicate violates the window condition, move the left side until the window is valid again.
- [] State the invariant: the current window contains no repeated character.
- [] State time and space complexity.
- [] Record an implementation mistake or observation.

Target complexity: $O(n)$ time with tracking space dependent on the character set/window representation.

9. DSA Problem 2 — Maximum Average Subarray I

Goal: Find the contiguous subarray of fixed length k with the maximum average.

Brute force

Calculate each candidate window's sum independently. Repeated work can make this $O(n \cdot k)$, which becomes $O(n^2)$ when k is proportional to n .

Optimized approach

1. Calculate the first window sum.
2. Slide one position.
3. Subtract the outgoing element.
4. Add the incoming element.
5. Track the maximum window sum.
6. Divide by k when returning the average.

Target complexity: $O(n)$ time and $O(1)$ extra space.

Invariant: At every position, the maintained sum represents exactly the current k -element window.

10. DSA Problem 3 — Minimum Size Subarray Sum

Goal: Find the smallest contiguous subarray whose sum is at least the target, under the problem assumptions.

Example:
target = 7
[2, 3, 1, 2, 4, 3]

A valid window:
[2, 3, 1, 2] = 8

Then shrink from the left while the condition remains satisfied.

Tasks

- [] Write the brute-force solution.
- [] Maintain a variable-size window.
- [] Expand right to increase the current sum.
- [] When the target condition is met, shrink left to find the smallest valid window.
- [] Track the best minimum length.
- [] State the invariant.
- [] Explain why the window can be shrunk instead of restarted.
- [] State time and space complexity.
- [] Record edge cases and one mistake/observation.

Important: The classic linear Sliding Window solution relies on the usual problem assumption that the numbers are positive/non-negative so expanding and shrinking have predictable effects on the sum.

11. Sliding Window vs Two Pointers

Pattern	Typical idea	Example
Two Pointers	Two indices exploit a relationship between positions/values	Valid Palindrome; Two Sum II
Sliding Window	Two boundaries maintain a contiguous active range	Longest Unique Substring; Maximum Average; Minimum Size Subarray Sum

Interview point: Sliding Window is often implemented with two boundaries, so it can be viewed as a specialized two-pointer technique. The defining idea is the maintained contiguous window and the rule controlling its expansion and contraction.

12. Block 6 — Git + Documentation

Time: 10 minutes

Recommended commits

```
feat: add document listing endpoint
feat: add document retrieval endpoint
feat: add document deletion endpoint
test: add document retrieval and deletion tests
feat: add sliding window DSA solutions
```

README update

AI Engineering Intelligence Platform

Current capabilities:

- Document API

Architecture:

API → Service → Repository → In-Memory Storage

Endpoints:

```
GET /
GET /health
POST /documents
GET /documents
GET /documents/{id}
DELETE /documents/{id}
```

Testing:

pytest

- ☐ Run git status.
- ☐ Review git log.
- ☐ Push changes.
- ☐ Verify GitHub source tree.
- ☐ Verify README.
- ☐ Verify tests.
- ☐ Verify commit history.

13. Day 4 End-of-Day Repository State

```
ai-engineering-intelligence-platform/
├── src/
│   ├── main.py
│   ├── api/
│   ├── services/
│   ├── models/
│   ├── repositories/
│   └── core/
├── tests/
├── docs/
├── requirements.txt
├── .gitignore
└── README.md
```

Expected API surface:

```
GET    /
GET    /health

POST   /documents
GET    /documents
GET    /documents/{id}
DELETE /documents/{id}
```

Follow your existing Day 2 codebase for exact internal file names and implementation details. Do not create unnecessary abstractions simply to make the tree look more sophisticated.

14. Day 4 Interview Objectives

By the end of Day 4, you should be able to answer these without notes:

- ☐ Why does GET /documents call DocumentService?
- ☐ Why should GET /documents/{id} map a missing document to 404?
- ☐ Why should DELETE /documents/{id} not directly manipulate storage?
- ☐ What is the difference between API schema, business object and database model?
- ☐ How does the repository boundary prepare the application for PostgreSQL?
- ☐ What happens end-to-end when a document is retrieved?
- ☐ How do API tests prove the behavior is repeatable?
- ☐ What is Sliding Window?
- ☐ How do you recognize a Sliding Window problem?
- ☐ What is the difference between fixed and variable windows?
- ☐ Why is Maximum Average Subarray I a fixed-window problem?
- ☐ Why are the other two Day 4 DSA problems variable-window problems?
- ☐ What is the invariant in each optimized DSA solution?
- ☐ Why is the optimized solution $O(n)$?
- ☐ What are common mistakes in pointer/window movement?
- ☐ Why is PostgreSQL deliberately postponed?

15. What NOT to Do on Day 4

Do not expand the stack merely because the project is called an AI engineering platform.

- ☐ Do not add PostgreSQL yet.
- ☐ Do not add Redis.
- ☐ Do not add Docker.
- ☐ Do not add Kubernetes.
- ☐ Do not add GCP.
- ☐ Do not add LangChain or LangGraph.
- ☐ Do not add embeddings.
- ☐ Do not add a vector database.
- ☐ Do not add RAG.
- ☐ Do not add agents.
- ☐ Do not add React.
- ☐ Do not add authentication.
- ☐ Do not add PDF-processing pipelines.

These technologies were deliberately postponed in the Day 2 foundation plan so that engineering fundamentals could be established before adding breadth.

16. Day 4 Definition of Done

Backend

- ☐ Existing Day 2 endpoints still work.
- ☐ POST /documents works.
- ☐ GET /documents works.
- ☐ GET /documents/{id} works.
- ☐ DELETE /documents/{id} works.
- ☐ Pydantic contracts are explicit.
- ☐ API → Service → Repository remains intact.
- ☐ Missing resources produce 404 behavior.
- ☐ In-memory storage is still used.
- ☐ Business rules remain outside route handlers.

Testing

- ☐ GET / tested.
- ☐ GET /health tested.
- ☐ POST /documents tested.
- ☐ GET /documents tested.
- ☐ GET /documents/{id} tested.
- ☐ Missing document tested.
- ☐ DELETE /documents/{id} tested.
- ☐ Invalid input tested.
- ☐ pytest passes.

DSA

- ☐ Sliding Window understood.
- ☐ Fixed vs variable window understood.
- ☐ Longest Substring Without Repeating Characters solved.
- ☐ Maximum Average Subarray I solved.
- ☐ Minimum Size Subarray Sum solved.
- ☐ Brute-force approaches recorded.
- ☐ Optimized approaches recorded.
- ☐ Time and space complexity recorded.
- ☐ Invariants understood.
- ☐ Mistakes/observations recorded.

Git

- ☐ Logical commits created.
- ☐ README updated.
- ☐ Changes pushed to GitHub.
- ☐ Repository state verified.

17. Day 4 Success Criteria

Day 4 is successful if you can move between implementation, architecture and interview explanation without relying on memorized scripts.

Backend maturity:

```
Minimal API
↓
Useful document API
↓
API → Service → Repository
↓
Automated tests
↓
Ready for database integration
```

DSA maturity:

```
Problem statement
↓
Pattern recognition
↓
Brute force
↓
Optimization
↓
Invariant
↓
Big-O
↓
Edge cases
↓
Interview explanation
```

Final target: Finish Day 4 with a backend that is still intentionally small, but whose boundaries are strong enough that Day 5 can introduce PostgreSQL as a persistence implementation rather than forcing a redesign of the entire application.

18. Day 4 Final Revision Checklist

- ☐ I can draw the complete API → Service → Repository architecture from memory.
- ☐ I can explain every endpoint without opening the code.
- ☐ I can explain why Pydantic models define the HTTP contract.
- ☐ I can distinguish API schema, business object and database model.
- ☐ I can explain the 404 path from application exception to HTTP response.
- ☐ I can explain how the API tests exercise the contract.
- ☐ I can explain why the repository boundary matters before PostgreSQL.
- ☐ I can recognize a Sliding Window problem.
- ☐ I can distinguish fixed-size and variable-size windows.
- ☐ I can solve all three Day 4 DSA problems without copying.
- ☐ I can state Big-O for all three.
- ☐ I can explain the invariant for all three.
- ☐ I can describe at least one edge case for each.
- ☐ I can explain why Day 4 does not add PostgreSQL yet.
- ☐ My GitHub repository contains clean commits, README and passing tests.

Day 4 → Day 5 transition: Once these objectives are complete, the next natural step is to replace or extend the in-memory persistence boundary with PostgreSQL while preserving the API and service contracts.